

实验3

Verilog时序逻辑设计

实验目的

- 1. 学习并掌握运用Verilog语言设计时序逻辑电路的原理和方法。
- 2. 掌握分频电路的设计方法；
- 3. 基于行为级描述实现8位移位寄存器；
- 4. 掌握7段数码管的译码及动态显示原理；
- 5. 基于行为级描述实现4位的十进制计数器；
- 6. 设计一个具有输入、输出、计时等功能的小型数字系统；

1. Verilog实现8位移位寄存器

输 入							输出
清零	控制信号		串行输入		时钟	并行输入	Q7~Q0
nclr	S1	S0	Dsr	Dsl	clk	D7~D0	
L	×	×	×	×	×	×	L
H	L	L	×	×	×	×	Q7~Q0
H	L	H	L	×	↑	×	L Q7~Q1
H	L	H	H	×	↑	×	H Q7~Q1
H	H	L	×	L	↑	×	Q6~Q0 L
H	H	L	×	H	↑	×	Q6~Q0 H
H	H	H	×	×	↑	D7~D0	D7~D0

- 时钟分频, 100MHz → 1 Hz
- 时钟相关约束文件修改

用Verilog构建寄存器和计数器

1位寄存器

为了设计一个1位寄存器，它可以在需要时从输入线in_data加载一个值，我们给D触发器增加一根输入线load，当我们想要从in_data加载一个值时，就把load设置为1，那么在下一个时钟上升沿，in_data的值将被存储在q中。

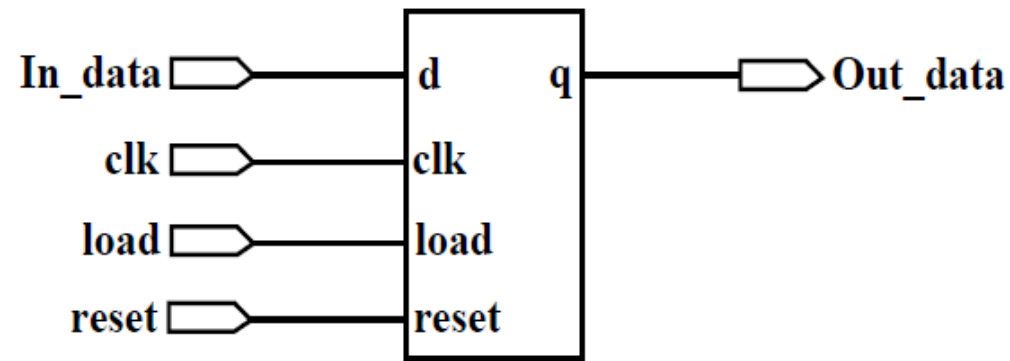


图4.12 1位寄存器逻辑符号

用Verilog构建寄存器和计数器

1位寄存器

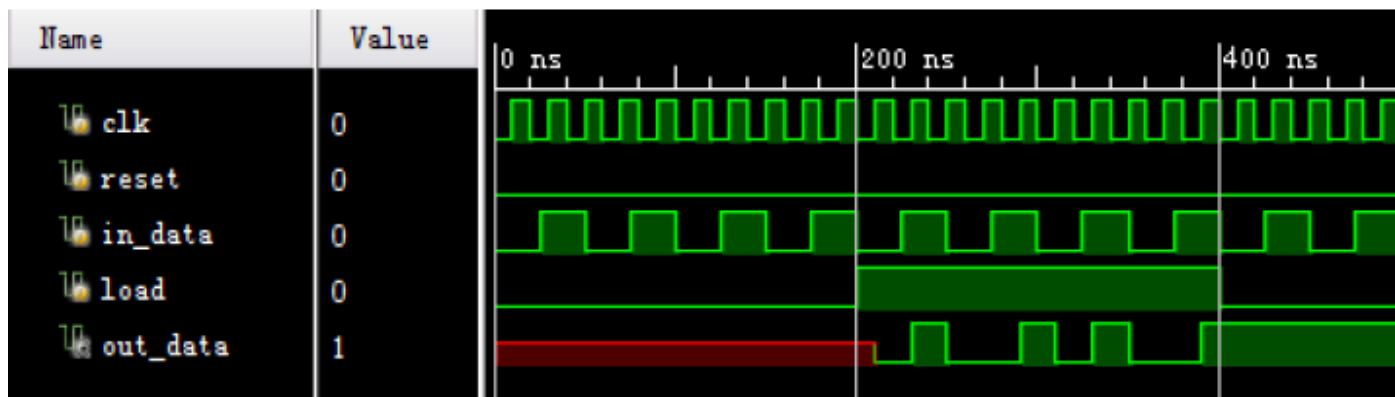


图4.13 1位寄存器的仿真时序图

从仿真结果我们可以看出:

- 当load信号为0时，输入线上的数据in_data就不会在每个时钟clk的上升沿被不断地重新加载到out_data，寄存器的输入out_data保持不变；
- 当load信号为1时，在下一个时钟clk的上升沿，out_data就变为in_data的值。

用Verilog构建寄存器和计数器

N位寄存器

- 如果我们把N个1位寄存器模块组合起来，就可以构成一个N位寄存器。
- 和1位寄存器不同之处在于，我们把输入in_data和out_data定义为一个N位的数组。

N位寄存器的逻辑符号如图4.14所示：

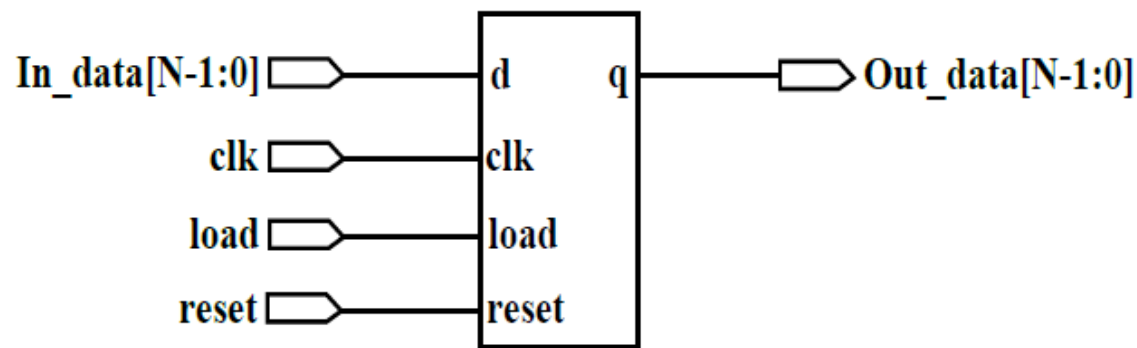


图4.14 1位寄存器逻辑符号



图4.15 8位寄存器的仿真时序图

用Verilog构建寄存器和计数器

N位寄存器

```
module reg_N
  #(parameter N = 8)
  (
    input clk,
    input reset,
    input [N-1:0] in_data,
    input load,
    output reg [N-1:0] out_data
  );
  always @(posedge clk, posedge reset)
    if(reset)
      out_data <= 0;
    else if(load == 1)
      out_data <= in_data;
endmodule
```

例 4.9 N位寄存器的描述

- 在例4.9中使用parameter语句，是为了使总线宽度可调。
- 默认状态下，总线宽度为8。

```
reg_N #(
  .N(16))
  fReg(.clk(clk),
    .reset(reset),
    .load(load),
    .in_data(indata),
    .out_data(out_data)
  );
```

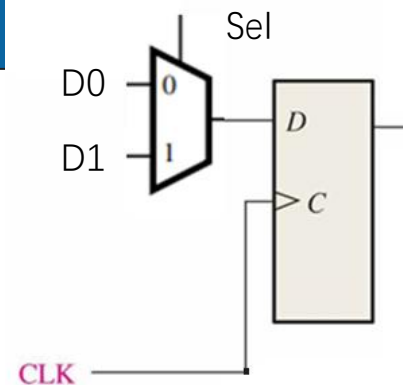
实例化1个16位宽度的寄存

用Verilog构建寄存器和计数器

移位寄存器

```
module shift4 (R, L, w, Clock, Q);  
  input [3:0] R;  
  input L, w, Clock;  
  output wire [3:0] Q;  
  
  muxdff Stage3 (w, R[3], L, Clock, Q[3]);  
  muxdff Stage2 (Q[3], R[2], L, Clock, Q[2]);  
  muxdff Stage1 (Q[2], R[1], L, Clock, Q[1]);  
  muxdff Stage0 (Q[1], R[0], L, Clock, Q[0]);  
  
endmodule
```

层次化4位移位寄存器



```
module muxdff (D0, D1, Sel, Clock, Q);  
  input D0, D1, Sel, Clock;  
  output reg Q;  
  
  always @(posedge Clock)  
    if (!Sel)  
      Q <= D0;  
    else  
      Q <= D1;  
  
endmodule
```

D输入端有一个2选1多路选择器的触发器

用Verilog构建寄存器和计数器

移位寄存器

```
module shift4 (R, L, w, Clock, Q);  
  input [3:0] R;  
  input L, w, Clock;  
  output wire [3:0] Q;  
  
  muxdff Stage3 (w, R[3], L, Clock, Q[3]);  
  muxdff Stage2 (Q[3], R[2], L, Clock, Q[2]);  
  muxdff Stage1 (Q[2], R[1], L, Clock, Q[1]);  
  muxdff Stage0 (Q[1], R[0], L, Clock, Q[0]);  
  
endmodule
```

层次化4位移位寄存器

```
module shift4 (R, L, w, Clock, Q);  
  input [3:0] R;  
  input L, w, Clock;  
  output reg [3:0] Q;  
  
  always @(posedge Clock)  
    if (L)  
      Q <= R;  
    else  
      begin  
        Q[0] <= Q[1];  
        Q[1] <= Q[2];  
        Q[2] <= Q[3];  
        Q[3] <= w;  
      end  
  
endmodule
```

4位移位寄存器的另一种代码

用Verilog构建寄存器和计数器

移位寄存器

```
module shiftn (R, L, w, Clock, Q);  
  parameter n = 16;  
  input [n-1:0] R;  
  input L, w, Clock;  
  output reg [n-1:0] Q;  
  integer k;  
  
  always @(posedge Clock)  
    if (L)  
      Q <= R;  
    else  
      begin  
        for (k = 0; k < n-1; k = k+1)  
          Q[k] <= Q[k+1];  
        Q[n-1] <= w;  
      end  
  
endmodule
```

n位移位寄存器

```
module shift4 (R, L, w, Clock, Q);  
  input [3:0] R;  
  input L, w, Clock;  
  output reg [3:0] Q;  
  
  always @(posedge Clock)  
    if (L)  
      Q <= R;  
    else  
      begin  
        Q[0] <= Q[1];  
        Q[1] <= Q[2];  
        Q[2] <= Q[3];  
        Q[3] <= w;  
      end  
  
endmodule
```

4位移位寄存器的另一种代码

用Verilog构建寄存器和计数器

移位寄存器

```
module shift_reg8
(
    input clk,
    input load,
    input [7:0] din,
    output qb
);
reg [7:0] reg8;
always @(posedge clk)
    if(load)
        reg8 <= din;
    else
        reg8[6:0] <= reg8[7:1];
    assign qb = reg8[0];
endmodule
```

例 4.11 具有同步预置功能的8位移位寄存器描述

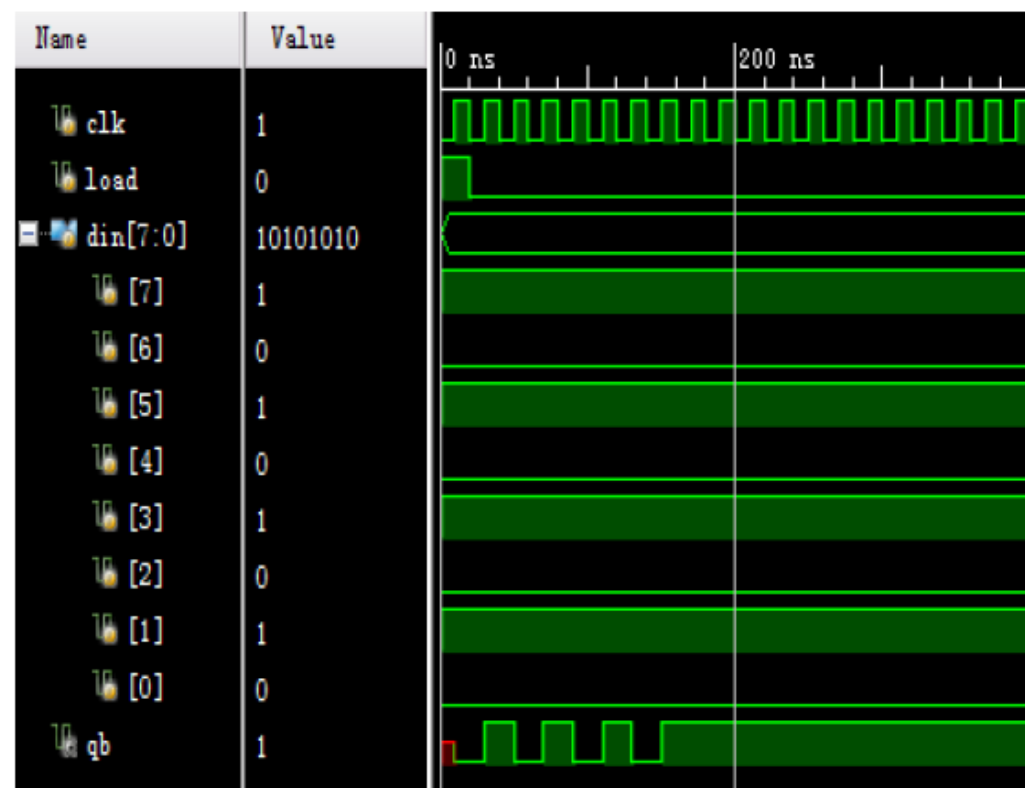


图4.16 8位寄存器的仿真时序图

用Verilog构建寄存器和计数器

8位通用移位寄存器

```
module univ_shift_reg
    #(parameter N = 8)
    (
        input clk, reset ;
        input [1:0] ctrl ;
        input [N-1:0] d ;
        output [N-1:0] q
    );
    //信号声明
    reg [N-1:0] r_reg, r_next;
    //寄存器
    always @(posedge clk, posedge reset)
```

```
    if (reset)
        r_reg <= 0;
    else
        r_reg <= r_next;           //next-state logic
    always @*
        case (ctrl)
            2'b00: r_next = r_reg;           //无操作
            2'b01: r_next = {r_reg[N-2:0], d[0]}; //左移
            2'b10: r_next = {d[N-1], r_reg[N-1:1]}; //右移
            default: r_next = d;             //载入
        endcase
    assign q = r_reg;               //输出逻辑
endmodule
```

例 4.12 8位通用移位寄存器描述

- 通用移位寄存器可以加载并行数据，将其内容向左移位、向右移位或保持原有状态，也可以实现并转串(第一次加载并行输入，然后移位)或串转并(首先移位，然后进行并行输出)。
- 可以把通用移位寄存器分为组合逻辑和时序逻辑两部分，各用一个always块来进行描述。下一状态逻辑使用4选1的多路选择器来选择寄存器所需下一状态的值。注意，d的最低位和最高位被用来作为串行输入的左移和右移操作。

用Verilog构建寄存器和计数器

计数器

```
module upcount (Resetn, Clock, E, Q);  
  input  Resetn, Clock, E;  
  output reg [3:0] Q;  
  
  always @(negedge Resetn, posedge Clock)  
    if (!Resetn)  
      Q <= 0;  
    else if (E)  
      Q <= Q + 1;  
  
endmodule
```

4位递增计数器

```
module upcount (R, Resetn, Clock, E, L, Q);  
  input  [3:0] R;  
  input  Resetn, Clock, E, L;  
  output reg [3:0] Q;  
  
  always @(negedge Resetn, posedge Clock)  
    if (!Resetn)  
      Q <= 0;  
    else if (L)  
      Q <= R;  
    else if (E)  
      Q <= Q + 1;  
  
endmodule
```

有并行载入端的4位递增计数器

用Verilog构建寄存器和计数器

计数器

```
module downcount (R, Clock, E, L, Q);  
    parameter n = 8;  
    input [n-1:0] R;  
    input Clock, L, E;  
    output reg [n-1:0] Q;  
  
    always @(posedge Clock)  
        if (L)  
            Q <= R;  
        else if (E)  
            Q <= Q - 1;  
  
endmodule
```

有并行载入端的4位递减计数器

```
module updowncount (R, Clock, L, E, up_down, Q);  
    parameter n = 8;  
    input [n-1:0] R;  
    input Clock, L, E, up_down;  
    output reg [n-1:0] Q;  
  
    always @(posedge Clock)  
        if (L)  
            Q <= R;  
        else if (E)  
            Q <= Q + (up_down ? 1 : -1);  
  
endmodule
```

N位递增/递减计数器

用Verilog构建寄存器和计数器

通用二进制计数器

```
module counter_univ_bin_N
#(parameter N = 8)
(
    input  clk,reset,load,up_down,
    input [N-1:0] d,
    output [N-1:0] qd
);
reg [N-1:0] regN;
always @(posedge clk)
    if (reset)
        regN <= 0;
    else if (load)
        regN <= d;
    else if (up_down)
        regN <= regN + 1;
    else regN <= regN - 1;
    assign qd = regN;
endmodule
```

例 4.14通用N位二进制计数器描述

通用二进制计数器具有更多功能：

可以实现增/减计数、暂停、预置初值，并有同步清0等功能。参数化的通用二进制计数器实现代码如例4.14所示。

计数器采用同步工作方式，同步时钟输入端口是clk。

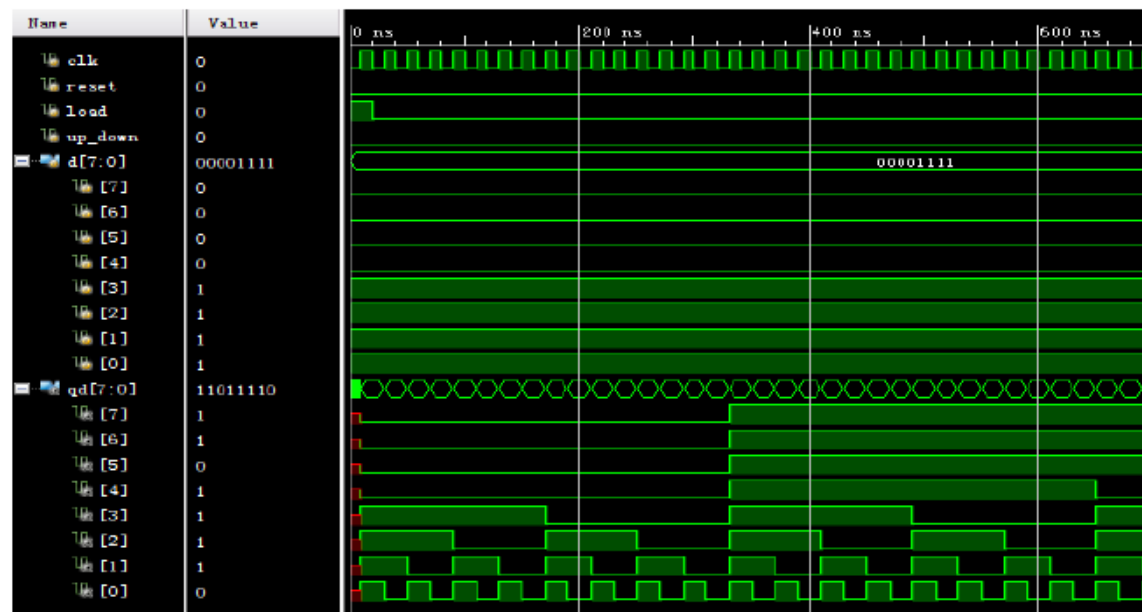


图4.18：简单8位二进制计数器的仿真时序图

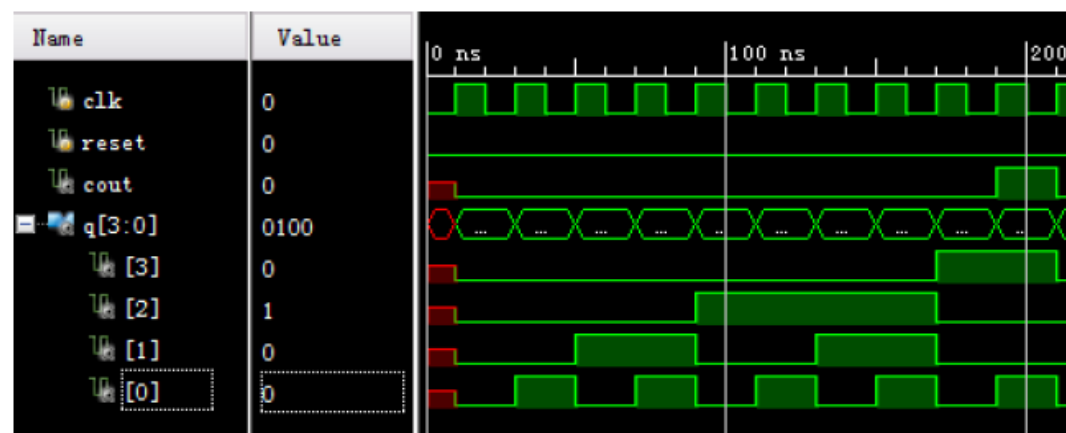
用Verilog构建寄存器和计数器

模m计数器

```
module counter_mod_m
#(parameter N = 4,           //计数器位数
  parameter M = 10)         //模M缺省为10
(
  input  clk,reset,
  output [N-1:0] qd,
  output cout
);
reg [N-1:0] regN;
always @(posedge clk)
  if(reset)
    regN <= 0;
  else if (regN < (M-1))
    regN <= regN + 1;
  else
    regN <= 0;
  assign qd = regN;
  assign cout = (regN == (M-1))?1'b1:1'b0;
endmodule
```

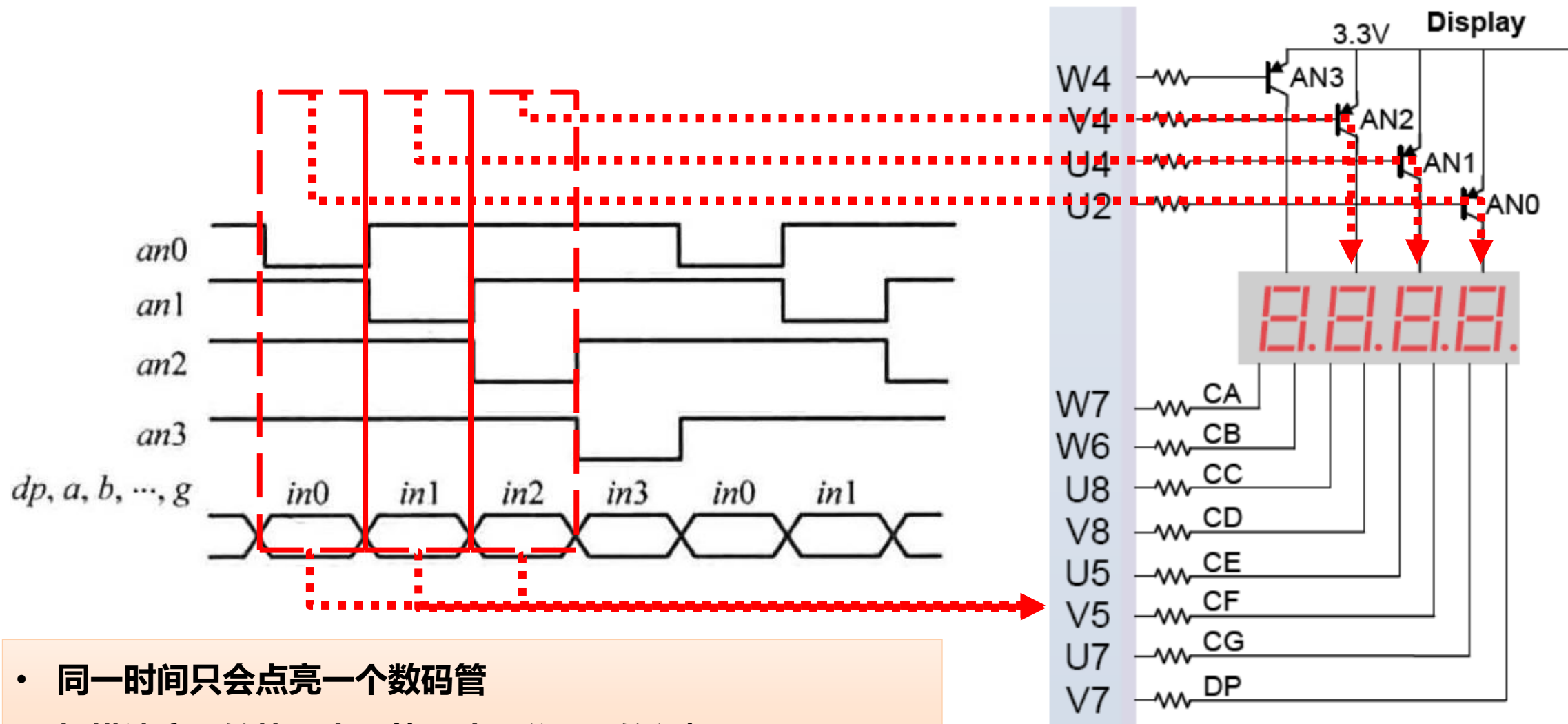
例 4.15 模m计数器描述

- 模m计数器的计数值从0增加到m-1，然后循环。参数化的模m计数器实现代码如例4.15所示。
- 它有两个参数：
参数M，它指定了计数模值m；
参数N，它指定了计数器所需的位数，等于 $\lceil \log_2 M \rceil$ 。
- 代码例4.15中计数器的默认值是模10，它的仿真工作波形如图4.19所示。



例 4.19 简单8位二进制计数器的仿真时序图

数码管扫描显示电路



- 同一时间只会点亮一个数码管
- 扫描速度足够快，人眼就无法区分LED的闪烁 ($\sim 1000\text{Hz}$)

数码管扫描显示电路

```

module scan_led_disp
(
    input clk,reset,
    input [7:0] in3,in2,in1,in0,
    output reg [3:0] an,
    output reg [7:0] sseg
);
    localparam N = 18; //对输入50MHz时钟进行分频
    (50 MHz/2^16)
    reg [N-1:0] regN;

    always @(posedge clk,posedge reset)
        if (reset)
            regN <= 0;
        else
            regN <= regN + 1;

```

```
always @*
  case (regN[N-1:N-2])
    2'b00: begin
      an = 4'b1110;
      sseg = in0;
    end
    2'b01: begin
      an = 4'b1101;
      sseg = in1;
    end
    2'b10: begin
      an = 4'b1011;
      sseg = in2;
    end
    default: begin
      an = 4'b0111;
      sseg = in3;
    end
  endcase
endmodule
```

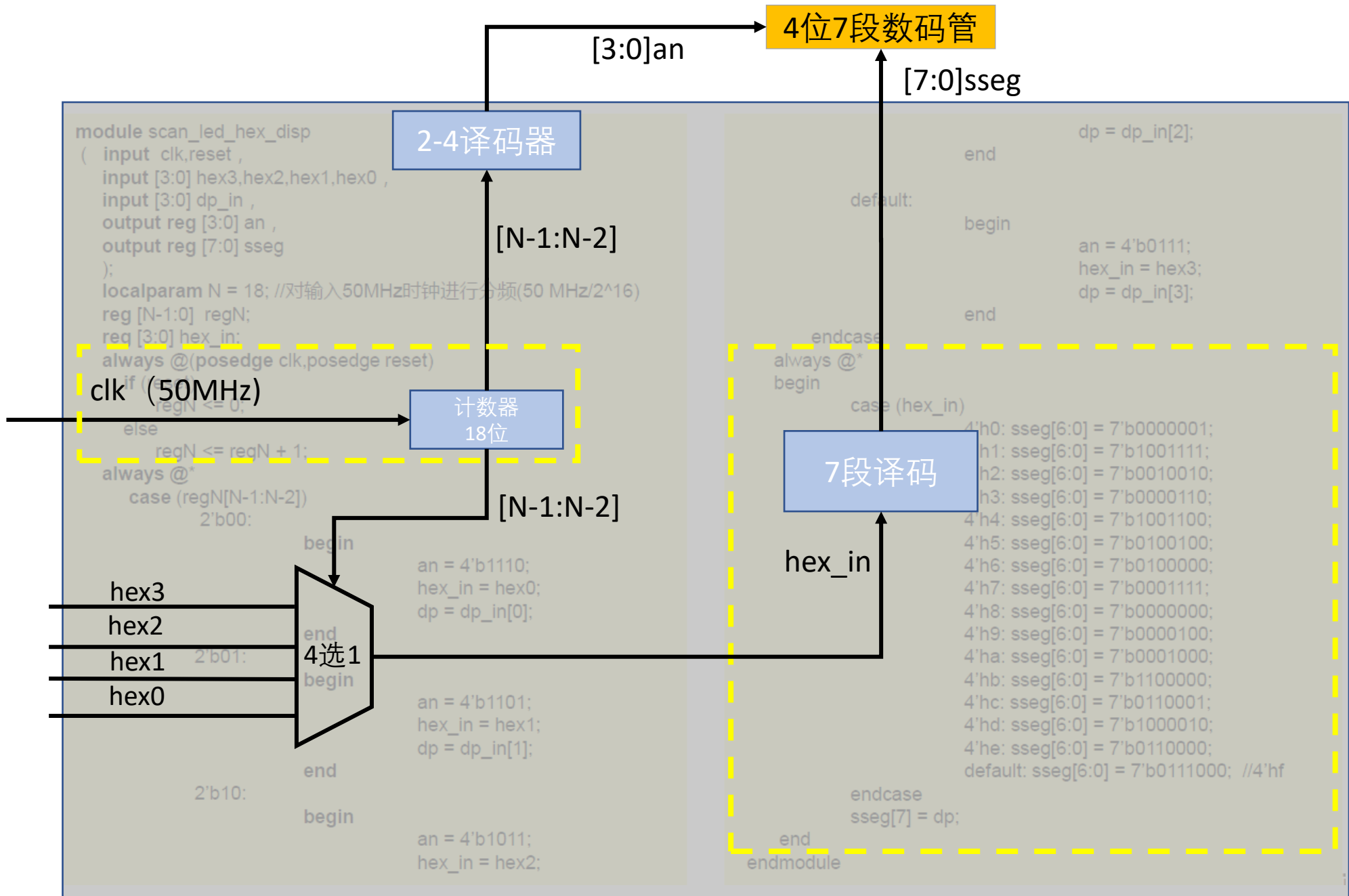
```

module scan_led_hex_disp
( input clk,reset ,
  input [3:0] hex3,hex2,hex1,hex0 ,
  input [3:0] dp_in ,
  output reg [3:0] an ,
  output reg [7:0] sseg
);
localparam N = 18; //对输入50MHz时钟进行分频(50 MHz/2^16)
reg [N-1:0] regN;
reg [3:0] hex_in;
always @(posedge clk,posedge reset)
  if (reset)
    regN <= 0;
  else
    regN <= regN + 1;
always @*
  case (regN[N-1:N-2])
    2'b00:
      begin
        an = 4'b1110;
        hex_in = hex0;
        dp = dp_in[0];
      end
    2'b01:
      begin
        an = 4'b1101;
        hex_in = hex1;
        dp = dp_in[1];
      end
    2'b10:
      begin
        an = 4'b1011;
        hex_in = hex2;

```

```

        dp = dp_in[2];
      end
    default:
      begin
        an = 4'b0111;
        hex_in = hex3;
        dp = dp_in[3];
      end
  endcase
always @*
begin
  case (hex_in)
    4'h0: sseg[6:0] = 7'b0000001;
    4'h1: sseg[6:0] = 7'b1001111;
    4'h2: sseg[6:0] = 7'b0010010;
    4'h3: sseg[6:0] = 7'b0000110;
    4'h4: sseg[6:0] = 7'b1001100;
    4'h5: sseg[6:0] = 7'b0100100;
    4'h6: sseg[6:0] = 7'b0100000;
    4'h7: sseg[6:0] = 7'b0001111;
    4'h8: sseg[6:0] = 7'b0000000;
    4'h9: sseg[6:0] = 7'b0000100;
    4'ha: sseg[6:0] = 7'b0001000;
    4'hb: sseg[6:0] = 7'b1100000;
    4'hc: sseg[6:0] = 7'b0110001;
    4'hd: sseg[6:0] = 7'b1000010;
    4'he: sseg[6:0] = 7'b0110000;
    default: sseg[6:0] = 7'b0111000; //4'hf
  endcase
  sseg[7] = dp;
end
endmodule
;
```



秒表与分频 (假设clk=50MHz)

```
module stop_watch
(input clk ,
input go,clr ,
output [3:0] d2,d1,d0;
);
localparam COUNT_VALUE = 5000000;
reg [22:0] ms_reg;
reg [3:0] d2_reg, d1_reg, d0_reg;
reg dp;
wire ms_tick;
reg [3:0] d2_next, d1_next, d0_next;
always @(posedge clk)
begin
    if (clr==0)
    begin
        ms_reg <= 23'b0;
        d2_reg <= 4'b0;
        d1_reg <= 4'b0;
        d0_reg <= 4'b0;
    end
end
```

```
else if (go == 1)
begin
    d2_reg <= d2_next;
    d1_reg <= d1_next;
    d0_reg <= d0_next;
    if (ms_reg < COUNT_VALUE-1)
        ms_reg <= ms_reg + 1;
    else
        ms_reg <= 23'b0;
end
end
assign ms_tick = (ms_reg == COUNT_VALUE-1) ? 1'b1
: 1'b0;
```

5. 如何让ms_tick占空比为50%?

1. ms_reg的计数序列是多少?

0, 1, 2,, COUNT_VALUE - 1

2. ms_tick的周期是多少?

ms_tick的周期是1/10s或100ms

3. ms_tick的脉冲宽度是多少?

1/50M s

4. 如何让ms_tick周期变成1ms?

COUNT_VALUE = 50 000

秒表

```
always @*
begin
    if (ms_tick)
        if (d0_reg != 9)
            d0_next = d0_reg + 1;
        else
            begin
                d0_next = 4'b0;
                if (d1_reg != 9)
                    d1_next = d1_reg + 1;
                else
                    begin
                        d1_next = 4'b0;
                        if (d2_reg != 9)
                            d2_next = d2_reg + 1;
                        else
                            d2_next = 4'b0;
                    end
                end
            end
        end
    end
end
```

```
assign d2 = d2_reg;

assign d1 = d1_reg;

assign d0 = d0_reg;

endmodule
```

例 4.19 秒表电路的Verilog HDL描述 (续)

时钟信号仿真

1. 使用always块和#延迟

这是最简单的方法，你可以直接在Verilog代码中定义一个always块，并使用#延迟来模拟时钟周期。

verilog

 复制代码

```
1 module clock_generator(  
2     output reg clk = 0  
3 );  
4     always #5 clk = ~clk; // 每5个时间单位反转clk，模拟时钟信号  
5 endmodule
```

verilog

 复制代码

```
1 module clock_generator(  
2     output reg clk = 0  
3 );  
4     initial begin  
5         forever #5 clk = ~clk; // 无限循环，每5个时间单位反转clk  
6     end  
7 endmodule
```

时钟信号仿真

verilog

复制代码

```
1 `timescale 1ns / 1ps
2
3 module counter_tb;
4
5 // 时钟和复位信号定义
6 reg clk = 0;
7 reg reset = 0;
8
9 // 实例化计数器模块
10 wire [7:0] count;
11 counter uut (
12     .clk(clk),
13     .reset(reset),
14     .count(count)
15 );
16
17 // 时钟信号生成
18 initial begin
19     // 初始化复位信号
20     reset = 1;
21     #10; // 等待10个时间单位
22     reset = 0; // 释放复位
23
24     // 生成时钟信号，每个时钟周期为10个时间单位
25     forever #10 clk = ~clk;
26 end
27
28 // 监视计数器输出
29 initial begin
30     $monitor("%d: count = %d", $time, count);
31     #100 $finish; // 仿真运行100个时间单位后结束
32 end
33
34 endmodule
```

在Verilog HDL中，`timescale`指令用于定义模块的仿真时的时间单位和时间精度。指令 `timescale 1ns / 1ps` 的含义如下：

- `1ns` 表示仿真时的时间单位是1纳秒（nanosecond）。这意味着当你在代码中指定一个延迟或时间值时，它是以纳秒为单位的。例如，`#200` 表示延时为200纳秒。
- `1ps` 表示仿真时的时间精度是1皮秒（picosecond）。精度决定了仿真中时间值的最小分辨率。因此，在这个设置中，你可以表示时间值如 `#200.123`，它指的是延时200.123纳秒。